

CSCE 465/765 – Computer & Network Security

Homework 2

Protect Agent Messages with Classic Cryptography

Report

Pradeep Periyasamy
Texas A&M University

October 4, 2026

Contents

1	Overview and Environment	2
2	Lab Preparation	2
3	Task 1 – Why Encryption Alone Is Insufficient (20 pts)	4
3.1	Design	4
3.2	Experiment	4
3.3	Root Cause	5
4	Task 2 – Authenticated Finite-Field Diffie–Hellman (30 pts)	6
4.1	Protocol	6
4.2	Checks before a session is accepted	7
4.3	Result	7
4.4	Connection to Textbook Concepts	7
4.5	MITM, Reflection, and Signature Analysis	8
5	Task 3 – Encrypt-then-MAC Record Layer (35 pts)	9
5.1	Format	9
5.2	How each requirement is met	9
5.3	Results	10
5.4	Finding: Counter-Block Overlap in the Assigned IV Layout	11
6	Task 4 – Adversarial Tests and Security Note (25 pts)	12
6.1	Required tests	12
6.2	Do the Tests Catch Real Bugs? (Mutation Check)	14
6.3	Security Note	17
7	How to Reproduce	17
8	AI Use	18
A	Captured Evidence (text)	19
A.1	Lab preparation	19
A.2	Task 1	19
A.3	Task 2	19
A.4	Task 3	20
A.5	Task 3 – OpenSSL check	20
A.6	Task 4 – pytest	20
A.7	Additional screenshots	22

1 Overview and Environment

All work was done inside my course VM (Ubuntu 24.04 on VirtualBox, NAT networking) in `~/csce465-agentsec/hw2`. Everything runs in one process: the gateway, the node and the on-path attacker are Python objects that pass byte strings to each other, so no network sockets are used, as the assignment allows. All primitives come from the `cryptography` library (AES-256-CTR, HMAC-SHA-256, SHA-256, RSA-PSS, finite-field DH); none of them are implemented by hand.

File	Purpose
<code>baseline_ctr.py</code>	Task 1 – AES-CTR without a MAC; bit-flip and replay attacks
<code>handshake.py</code>	Task 2 – RSA-PSS-authenticated <code>ffdhe3072</code> handshake and the assigned key derivation
<code>secure_record.py</code>	Task 3 – encrypt-then-MAC record layer: <code>seal()</code> / <code>open_record()</code>
<code>tests/</code>	Task 4 – 49 pytest tests (handshake, record layer, baseline)
<code>ffdhe3072.pem</code>	RFC 7919 group generated with OpenSSL in Lab Preparation
<code>inspect_record.py</code>	Dumps one record and its keys for an independent OpenSSL check
<code>capture_evidence.sh</code>	Regenerates every file in <code>evidence/</code>

2 Lab Preparation

```
python3 -m venv ../.venv && source ../.venv/bin/activate
python -m pip install cryptography==49.0.0 pytest==9.1.1
openssl genpkey -genparam -algorithm DH -pkeyopt group:ffdhe3072 -out ffdhe3072.pem
```

```

ntu Tamu [Running] - Oracle VirtualBox
Oct 4 17:49
pradeep@pradeep: ~/csce465-agentsec/hw2
(.venv) pradeep@pradeep:~/csce465-agentsec/hw2$ python3 --version
python3 3.12.3
pradeep@pradeep:~/csce465-agentsec/hw2$ openssl version
openssl 3.0.13 30 Jan 2024 (Library: OpenSSL 3.0.13 30 Jan 2024)
pradeep@pradeep:~/csce465-agentsec/hw2$ pip show cryptography pytest
Name: cryptography
Version: 49.0.0
Summary: cryptography is a package which provides cryptographic recipes and primitives to Python developers.
Home-page: https://github.com/pyca/cryptography
Author:
Author-email: The Python Cryptographic Authority and individual contributors <cryptography-dev@python.org>
License-Expression: Apache-2.0 OR BSD-3-Clause
Location: /home/pradeep/csce465-agentsec/.venv/lib/python3.12/site-packages
Requires: cffi
Required-by:
---
Name: pytest
Version: 9.1.1
Summary: pytest: simple powerful testing with Python
Home-page: https://docs.pytest.org/en/latest/
Author: Brianna Laughner, Bruno Oliveira, Floris Bruynooghe, Freya Bruhin, Holger Krekel, Others (See AUTHORS), Ronny Pfannschmidt
Author-email:
License-Expression: MIT
Location: /home/pradeep/csce465-agentsec/.venv/lib/python3.12/site-packages
Requires: iniconfig, packaging, pluggy, pygments
Required-by:
DH Parameters: (3072 bit)
GROUP: ffdhe3072
(.venv) pradeep@pradeep:~/csce465-agentsec/hw2$

```

Figure 1: Tool versions inside the venv and the ffdhe3072 group check.

Item	Value on my VM
Python	3.12.3
OpenSSL	3.0.13 (30 Jan 2024)
cryptography	49.0.0
pytest	9.1.1
Group file	DH Parameters: (3072 bit), GROUP: ffdhe3072

On a fresh Ubuntu 24.04 install, `python3 -m venv` first failed because `ensurepip` was missing; installing `python3.12-venv` fixed it.

3 Task 1 – Why Encryption Alone Is Insufficient (20 pts)

3.1 Design

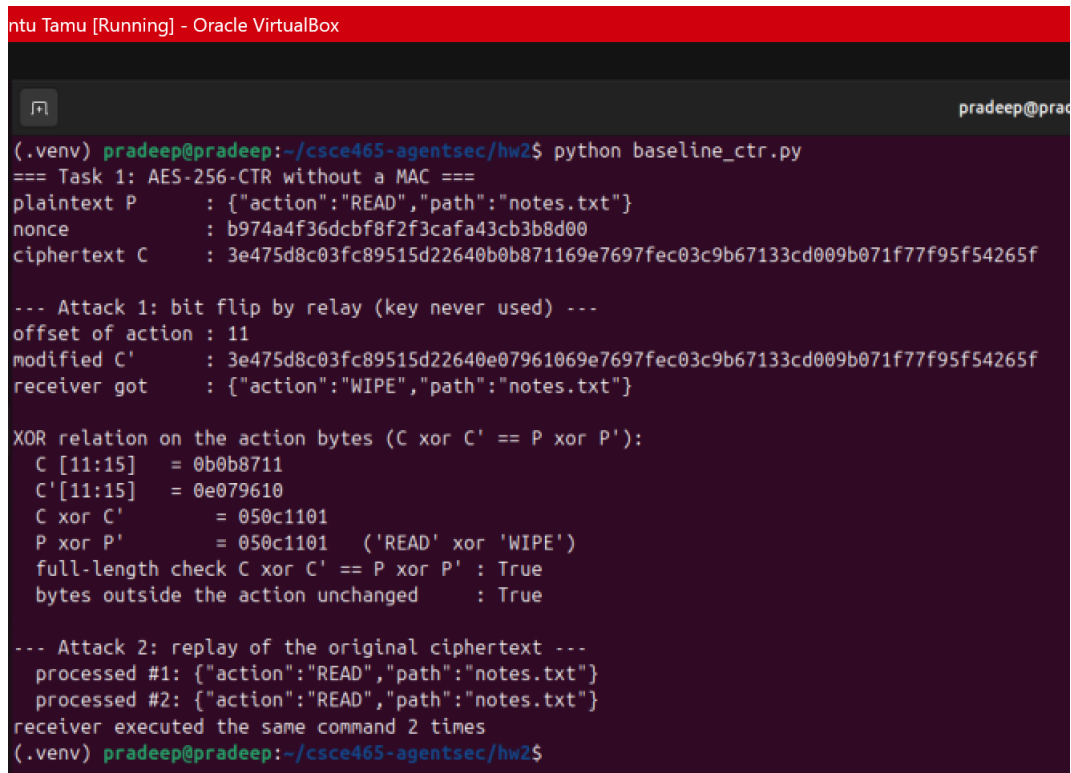
The baseline has two endpoints, **Sender** and **Receiver**, that share a 256-bit key K . The sender encrypts the fixed command

```
{"action":"READ","path":"notes.txt"}
```

with AES-256-CTR under a fresh 16-byte random nonce. This baseline has no Message Authentication Code (MAC) and no sequence number. The receiver decrypts the ciphertext, parses the JSON and executes the operation. An on-path relay, implemented as `relay_flip()`, can read, change or resend the bytes in transit, but never has K .

Because the command has a fixed JSON layout, the attacker knows without decrypting anything that the value **READ** sits at byte offsets 11–14. This is a realistic assumption: tool-call formats are usually public. For the bit-flipping attack, the relay takes the ciphertext C and computes the mask $\Delta = \text{'READ'} \oplus \text{'WIPE'} = 05\ 0c\ 11\ 01$. XORing Δ into $C[11:15]$ changes the decrypted plaintext without breaking the surrounding JSON. For the replay attack, the relay keeps a copy of a valid ciphertext and sends it to the receiver again later.

3.2 Experiment



```
ntu Tamu [Running] - Oracle VirtualBox

pradeep@pradeep:~/csce465-agentsec/hw2$ python baseline_ctr.py
=== Task 1: AES-256-CTR without a MAC ===
plaintext P      : {"action":"READ","path":"notes.txt"}
nonce           : b974a4f36dcfb8f2f3cafa43cb3b8d00
ciphertext C     : 3e475d8c03fc89515d22640b0b871169e7697fec03c9b67133cd009b071f77f95f54265f

--- Attack 1: bit flip by relay (key never used) ---
offset of action : 11
modified C'      : 3e475d8c03fc89515d22640e07961069e7697fec03c9b67133cd009b071f77f95f54265f
receiver got     : {"action":"WIPE","path":"notes.txt"}

XOR relation on the action bytes (C xor C' == P xor P'):
C [11:15]       = 0b0b8711
C' [11:15]      = 0e079610
C xor C'        = 050c1101
P xor P'        = 050c1101 ('READ' xor 'WIPE')
full-length check C xor C' == P xor P' : True
bytes outside the action unchanged      : True

--- Attack 2: replay of the original ciphertext ---
processed #1: {"action":"READ","path":"notes.txt"}
processed #2: {"action":"READ","path":"notes.txt"}
receiver executed the same command 2 times
(.venv) pradeep@pradeep:~/csce465-agentsec/hw2$
```

Figure 2: python baseline_ctr.py: bit flip READ→WIPE without the key, then replay.

Observation (Figure 2)	Value
Original plaintext	<code>{"action": "READ", "path": "notes.txt"}</code>
Receiver decrypted after flip	<code>{"action": "WIPE", "path": "notes.txt"}</code>
$C[11:15] \rightarrow C'[11:15]$	<code>0b0b8711</code> $\rightarrow$ <code>0e079610</code>
$C \oplus C'$	<code>050c1101</code>
$P \oplus P'$ ('READ' $\oplus$ 'WIPE')	<code>050c1101</code>
Full-length check $C \oplus C' = P \oplus P'$	True; every byte outside the action field unchanged
Replay	Same ciphertext processed twice ("executed the same command 2 times")

3.3 Root Cause

Both attacks come from how CTR mode works and from the missing integrity and freshness checks. AES-CTR builds a keystream $KS = \text{AES}_K(\text{nonce} \parallel \text{ctr})$ and produces $C = P \oplus KS$; decryption computes $P = C \oplus KS$. Since the keystream is simply XORed with the message, adding a mask Δ to the ciphertext gives

$$C' \oplus KS = (C \oplus \Delta) \oplus KS = (P \oplus KS \oplus \Delta) \oplus KS = P \oplus \Delta.$$

This property is called malleability: the attacker can change chosen plaintext bits in a fully predictable way without knowing K . All the attacker needs is the structure of the plaintext and the offset of the field. Because **READ** and **WIPE** have the same length, the result is still valid JSON, so the receiver parses and executes the tampered command without noticing.

The replay attack shows the difference between confidentiality, integrity and freshness. Encryption alone only hides the content from a passive eavesdropper. It does not tie a ciphertext to a sequence number, a time or a session, so the receiver cannot tell an old message that was resent from a new instruction. Fixing this needs a MAC (I use encrypt-then-MAC) together with explicit sequence numbers, which is what Task 3 adds.

4 Task 2 – Authenticated Finite-Field Diffie–Hellman (30 pts)

4.1 Protocol

Gateway (initiator) and node (responder) each hold a long-term RSA-3072 key and know the peer's public key and identity in advance. Every session uses fresh `ffdhe3072` private values and fresh 16-byte nonces. Three length-prefix-encoded messages:

Msg	Dir.	Contents
HS1	G→N	"HS1", ID_G , g^x (384 B), n_G (16 B)
HS2	N→G	"HS2", ID_N , g^y (384 B), n_N (16 B), $\text{Sig}_N(\text{"node"} \parallel TH)$
HS3	G→N	"HS3", $\text{Sig}_G(\text{"gateway"} \parallel TH)$

Canonical transcript. Each field is a 4-byte big-endian length followed by the bytes, in exactly this order:

$$\begin{aligned} transcript = & LP(\text{CSCE465-HS-v2}) \parallel LP(\text{ffdhe3072}) \parallel LP(ID_G) \parallel LP(ID_N) \\ & \parallel LP(g^x) \parallel LP(g^y) \parallel LP(n_G) \parallel LP(n_N), \quad TH = \text{SHA-256}(transcript) \end{aligned}$$

Fields are ordered by *role* (gateway first), not by who sent them, so both sides build byte-identical transcripts. `transcript_hash()` parses first and rejects a wrong declared length, field count, label, group or field size *before* SHA-256 runs. A test patches the hash function and asserts that it is never called on a malformed transcript.

Signatures. RSA-PSS (MGF1-SHA-256, 32-byte salt) over $role \parallel TH$. The gateway verifies the node's signature before using the DH secret; the node accepts only after verifying the gateway's signature in HS3.

Key derivation (as assigned). Z is the DH shared secret as a 384-byte, left-zero-padded big-endian string. DH public values are encoded the same way.

```

K_master   = SHA-256("CSCE465-KDF-v1" || Z || TH)
K_g2n_enc  = HMAC-SHA-256(K_master, "gateway-to-node encryption" || TH)
K_g2n_mac  = HMAC-SHA-256(K_master, "gateway-to-node MAC" || TH)
K_n2g_enc  = HMAC-SHA-256(K_master, "node-to-gateway encryption" || TH)
K_n2g_mac  = HMAC-SHA-256(K_master, "node-to-gateway MAC" || TH)
session_id = first 8 bytes of HMAC-SHA-256(K_master, "session identifier" || TH)

```

A dedicated test (`test_kdf_matches_assignment_spec_independently`) recomputes all six values with Python's standard-library `hashlib/hmac` and compares them.

4.2 Checks before a session is accepted

Check	Error raised	Example test
Strict TLV parsing (truncated, trailing, wrong count)	MalformedMessage	test_truncated_*
Message type matches protocol state	MalformedMessage/ProtocolStateError	test_out_of_order_*
Peer identity equals expected identity	UnexpectedIdentity	test_unexpected_peer_*
Peer did not echo my nonce or DH value	ReflectionDetected	test_reflected_dh_*
DH public value in $[2, p - 2]$	InvalidPublicValue	test_degenerate_dh_*
RSA-PSS signature over $role \parallel TH$	BadSignature	test_invalid_pss_*
Group file prime = RFC 7919 ffdhe3072	ValueError	load_group()

4.3 Result

```

ntu Tamu [Running] - Oracle VirtualBox
(.venv) pradeep@pradeep:~/csce465-agentsec/hw2$ python handshake.py
=== Task 2: authenticated ffdhe3072 handshake ===
group: ffdhe3072, p bits = 3072, g = 2
  HS1: 429 bytes on the wire
  HS2: 814 bytes on the wire
  HS3: 395 bytes on the wire
gateway state=established, node state=established
TH      = 629021a3cfe9ac94017d63853042fdb82ad62f5b22086944eba2cf43ddee9e8d
session_id = 169dc0cba38ce5fd
both sides derived identical keys: True
4 traffic keys pairwise distinct: True
second session (same RSA keys) -> new session_id 6748496128f6f3f7, keys differ: True
(.venv) pradeep@pradeep:~/csce465-agentsec/hw2$

```

Figure 3: python handshake.py: both sides established with identical keys; a second session gets new keys.

4.4 Connection to Textbook Concepts

The handshake design maps onto the textbook concepts as follows:

- **Diffie–Hellman key establishment:** the two parties agree on a shared secret $Z = g^{xy} \bmod p$ over an insecure channel. Using the standardized RFC 7919 3072-bit group (**ffdhe3072**) means I don't choose parameters myself, and computing discrete logarithms in this group is infeasible for a classical attacker.

- **Entity authentication with digital signatures:** plain DH says nothing about who is on the other side. Each party signs $role \parallel TH$ with RSA-PSS, which proves it holds its long-term private key and that it saw exactly the same transcript as its peer.
- **Freshness and replay resistance:** each party contributes a fresh 16-byte random nonce (n_G, n_N) that goes into TH . Every session therefore has a unique transcript, and a signature from an earlier handshake cannot be replayed into a new one.
- **Forward secrecy:** the long-term RSA keys are used only for signing. The ephemeral DH values x, y are generated per session and discarded after the handshake (`_forget_ephemeral()`). If an RSA private key leaks later, it cannot be used to recompute the Z of past sessions.
- **Key separation:** Z is never used directly. A master key K_{master} is expanded with different labels into four direction-specific keys (encryption and MAC for each direction) plus an 8-byte session identifier, so a key used for one purpose or direction is never reused for another.
- **Canonical length-prefixed encoding:** plain concatenation is ambiguous ("`gw`" + "`node1`" gives the same bytes as "`gwn`" + "`ode1`"). The 4-byte length prefixes make every transcript parse in exactly one way, so two different sets of fields can never produce the same signed bytes. The fixed role-based field order is what makes both sides compute the same transcript.

4.5 MITM, Reflection, and Signature Analysis

- **Man-in-the-middle resistance:** if an active attacker (Mallory) changes g^y or n_N in transit, the TH computed by the gateway differs from the TH the node signed, so RSA-PSS verification fails (`test_changed_dh_public_value`, `test_changed_nonce`). Mallory cannot produce a valid signature over the modified TH because she does not have the node's private key (`test_mitm_with_own_rsa_key`).
- **Identity binding:** putting both identities (ID_G, ID_N) into the transcript prevents identity misbinding (unknown key-share attacks): a signature made for a session with one peer cannot be redirected into a session with another.
- **Reflection resistance:** each signature covers an explicit role string ("`gateway`" or "`node`"), so a party's own message reflected back to it does not verify. With distinct RSA keys the peer-key check already stops this, but the role prefix is essential when one entity uses the same RSA key in both roles. My mutation test confirms this: removing the role prefix makes `test_reflection_with_shared_identity_key_rejected` fail. Separate checks for echoed nonces, echoed DH values and wrong message types handle the other reflection cases.
- **Without signatures:** the protocol becomes anonymous Diffie–Hellman. Mallory can run one DH exchange with the gateway and another with the node, obtaining Z_{GM} and Z_{MN} , and then decrypt, read, modify and re-encrypt all record-layer traffic without being noticed. The record MACs still verify, because they use the keys Mallory derived. Record protection cannot fix a key exchange that was never authenticated.

5 Task 3 – Encrypt-then-MAC Record Layer (35 pts)

5.1 Format

```

header    = version(1) || direction(1) || sequence(8) || message_type(1) || ct_len(4)    (15
  bytes)
iv        = session_id(8) || sequence(8)
ciphertext = AES-256-CTR(K_enc, iv, plaintext)
tag       = HMAC-SHA-256(K_mac, header || iv || ciphertext)
record    = header || ciphertext || tag

```

The IV is not sent on the wire. Both sides rebuild it from `session_id` and the sequence number, and it is covered by the tag. Direction `0x01` = gateway→node, `0x02` = node→gateway.

Receive order in `open_record()`: framing → version → direction → exact expected sequence → HMAC verify (library, constant-time) → AES-CTR decrypt → advance the sequence. Any failure raises; no plaintext is returned and receiver state is unchanged.

5.2 How each requirement is met

Requirement	Implementation
Separate keys per direction	<code>channel_states()</code> : gateway sends with <code>g2n</code> keys and receives with <code>n2g</code> keys; the node gets the reverse.
Sequence starts at 0 per direction	Each Send/Recv state starts at 0; both headers show seq 00...00 (Fig. 4).
Sender never reuses an IV under one key	seq increments after every <code>seal()</code> ; at 2^{64} the channel closes (<code>SequenceExhausted</code>) instead of wrapping.
Receiver expects exact next sequence	mismatch → <code>ReplayOrReorder</code> (covers replay, reorder, drop).
Verify HMAC before decrypting	decrypt is reached only after <code>verify()</code> succeeds; a test asserts the cipher never runs on a bad record.
Reject modified header/ciphertext, replay, wrong direction	<code>BadMAC</code> / <code>BadMAC</code> / <code>ReplayOrReorder</code> / <code>WrongDirection</code> .
No plaintext after any error	every check raises before decryption.
Constant-time MAC verification	<code>cryptography...hmac.HMAC.verify()</code> , not <code>==</code> .

5.3 Results

```
ntu Tamu [Running] - Oracle VirtualBox

pradeep@pradeep:~/csce465-agentsec/hw2$ python secure_record.py
=== Task 3: encrypt-then-MAC record layer ===
G->N record #0: 83 bytes, header=010100000000000000000002000000024
  node opened : (32, b'{"action":"READ","path":"notes.txt"}')
N->G record #0: 73 bytes, header=01020000000000000000000210000001a
  gateway opened: (33, b'{"ok":true,"data":"hello"}')
  replay G->N #0: rejected -> ReplayOrReorder: got sequence 0, expected 1
  reflect N->G record back to node: rejected -> WrongDirection: got direction 2, expected 1
  Task-1 bit-flip on protected record: rejected -> BadMAC
(.venv) pradeep@pradeep:~/csce465-agentsec/hw2$
```

Figure 4: python secure_record.py: both directions work; replay, reflection and the Task 1 bit flip are rejected.

Independent check with OpenSSL (not my Python code): the ciphertext decrypts with `openssl enc -aes-256-ctr`, and `openssl dgst -hmac` over *header* || *iv* || *ct* gives exactly the record's tag.

```
ntu Tamu [Running] - Oracle VirtualBox

pradeep@pradeep: ~/csce465
Oct 4 17:52

(.venv) pradeep@pradeep:~/csce465-agentsec/hw2$ python inspect_record.py > /tmp/v.env && . /tmp/v.env
hex2bin(){ python3 -c "import sys;sys.stdout.buffer.write(bytes.fromhex(sys.argv[1]))" "$1"; }
echo "HDR=$HDR IV=$IV"
hex2bin $CT | openssl enc -d -aes-256-ctr -K $ENC -iv $IV; echo
hex2bin "$HDR$IV$CT" | openssl dgst -sha256 -mac HMAC -macopt hexkey:$MAC | awk '{print $NF}'
echo $TAG
HDR=010100000000000000000002000000024 IV=02f16629102bf75c0000000000000000
{"action":"READ","path":"notes.txt"}
ef6902e02aad959d7c9f3fbbe297adc7aba576586dab1b143cfc7214b93e5428
ef6902e02aad959d7c9f3fbbe297adc7aba576586dab1b143cfc7214b93e5428
(.venv) pradeep@pradeep:~/csce465-agentsec/hw2$
```

Figure 5: OpenSSL decrypts the record and reproduces the identical HMAC tag.

Field (Figure above)	Value
version / direction	01 / 01 (gateway→node)
sequence	0000000000000000
message_type / ct_len	20 (tool call) / 00000024 (36 bytes)
iv	02f16629102bf75c 0000000000000000
OpenSSL HMAC = tag	ef6902e0...3e5428 (identical)

5.4 Finding: Counter-Block Overlap in the Assigned IV Layout

AES-CTR starts the 128-bit counter block at the IV and increments the whole 128-bit value for every further 16-byte block. In the assigned format, $iv = session_id \parallel seq$, so the 64-bit sequence number sits in the low 64 bits. Encrypting a record of $m > 1$ blocks therefore uses counter values seq through $seq + m - 1$ in the low half. As a result, block 1 of record n uses the counter $session_id \parallel (seq + 1)$, which is exactly block 0 of record $n + 1$.

Impact and fix. Every starting IV is unique, but CTR actually needs every counter *block* to be unique under the same key. When two blocks share a counter value, they share keystream:

$$C_{n,1} \oplus C_{n+1,0} = (P_{n,1} \oplus KS) \oplus (P_{n+1,0} \oplus KS) = P_{n,1} \oplus P_{n+1,0}.$$

This is a two-time pad: the XOR of plaintext blocks from consecutive records leaks, which hurts confidentiality. Integrity is not affected, because encrypt-then-MAC still prevents forgery. To follow the assignment exactly, I kept the specified wire format and pinned this behavior with `test_spec_iv_layout_counter_overlap_is_documented`. In a real deployment it could be fixed by moving the sequence number into the high 64 bits, by using a GCM-style layout with an explicit 32-bit (4-byte) block counter, $iv = sid(4) \parallel seq(8) \parallel ctr(4)$, or by deriving a separate key per record.

6 Task 4 – Adversarial Tests and Security Note (25 pts)

6.1 Required tests

Each test asserts a *specific* exception type and, where relevant, that no keys or plaintext were produced and the receiver state did not advance.

Required case	Test(s)	Asserted failure
1. Valid handshake + bidirectional	<code>test_valid_handshake_both_sides_agree,</code> <code>test_valid_handshake_and_bidirectional_messages</code>	keys equal; seq 0,1,2 both ways
2. Modified ciphertext	<code>test_modified_ciphertext_rejected,</code> <code>test_task1_bitflip_attack_rejected,</code> <code>test_modified_tag_rejected</code>	BadMAC
3. Modified authenticated header	<code>test_modified_message_type_rejected</code> (+ version, length, sequence)	BadMAC / MalformedRecord / ReplayOrReorder
4. Replayed record	<code>test_replayed_record_rejected,</code> <code>test_reordered_or_dropped_record_rejected</code>	ReplayOrReorder
5. Reflected record	<code>test_reflected_record_rejected,</code> <code>test_reflected_record_with_rewritten_direction_rejected</code>	WrongDirection / BadMAC
6. Wrong RSA key / invalid PSS / reflected handshake	<code>test_incorrect_rsa_public_key_rejected,</code> <code>test_invalid_pss_signature_rejected,</code> <code>test_reflected_signature_rejected_by_role_binding,</code> <code>test_reflection_with_shared_identity_key_rejected</code>	BadSignature

Additional tests: MITM with an attacker RSA key; changed nonces and DH values in HS1/HS2; degenerate DH values $(0, 1, p-1, p)$; unexpected identity; malformed/truncated handshake messages; transcript rejected before hashing; KDF checked against the standard library; wire format checked byte-for-byte; record from another session; verify-before-decrypt; IV uniqueness and sequence exhaustion.

```

ntu Tamu [Running] - Oracle VirtualBox
Oct 4 17:53
pradeep@pradeep:~/csce465-agentsec/hw2

(.venv) pradeep@pradeep:~/csce465-agentsec/hw2$ python -m pytest -v
===== test session starts =====
platform linux -- Python 3.12.3, pytest-9.1.1, pluggy-1.6.0 -- /home/pradeep/csce465-agentsec/.venv/bin/python
cachedir: .pytest_cache
rootdir: /home/pradeep/csce465-agentsec/hw2
collected 49 items

tests/test_baseline.py::test_bitflip_changes_action_without_key PASSED
tests/test_baseline.py::test_replay_is_processed_twice PASSED
tests/test_handshake.py::test_valid_handshake_both_sides_agree PASSED
tests/test_handshake.py::test_every_session_uses_fresh_dh_and_nonces PASSED
tests/test_handshake.py::test_kdf_matches_assignment_spec_independently PASSED
tests/test_handshake.py::test_values_are_384_byte_left_padded PASSED
tests/test_handshake.py::test_transcript_is_length_prefixed_and_unambiguous PASSED
tests/test_handshake.py::test_incorrect_rsa_public_key_rejected PASSED
tests/test_handshake.py::test_invalid_pss_signature_rejected PASSED
tests/test_handshake.py::test_mitm_with_own_rsa_key_rejected PASSED
tests/test_handshake.py::test_changed_nonce_rejected[HS1-3] PASSED
tests/test_handshake.py::test_changed_nonce_rejected[HS2-3] PASSED
tests/test_handshake.py::test_changed_dh_public_value_rejected[HS1] PASSED
tests/test_handshake.py::test_changed_dh_public_value_rejected[HS2] PASSED
tests/test_handshake.py::test_degenerate_dh_public_value_rejected[0] PASSED
tests/test_handshake.py::test_degenerate_dh_public_value_rejected[1] PASSED
tests/test_handshake.py::test_degenerate_dh_public_value_rejected[p-1] PASSED
tests/test_handshake.py::test_degenerate_dh_public_value_rejected[p] PASSED
tests/test_handshake.py::test_unexpected_peer_identity_rejected PASSED
tests/test_handshake.py::test_reflected_signature_rejected_by_role_binding PASSED
tests/test_handshake.py::test_reflection_with_shared_identity_key_rejected PASSED
tests/test_handshake.py::test_reflected_dh_value_and_nonce_rejected PASSED
tests/test_handshake.py::test_reflected_hs1_back_to_gateway_rejected PASSED
tests/test_handshake.py::test_transcript_with_bad_declared_length_rejected_before_hashing PASSED
tests/test_handshake.py::test_truncated_handshake_message_rejected[1] PASSED
tests/test_handshake.py::test_truncated_handshake_message_rejected[3] PASSED
tests/test_handshake.py::test_truncated_handshake_message_rejected[50] PASSED
tests/test_handshake.py::test_trailing_bytes_rejected PASSED
tests/test_handshake.py::test_out_of_order_message_rejected PASSED
tests/test_record.py::test_valid_handshake_and_bidirectional_messages PASSED
tests/test_record.py::test_empty_plaintext_roundtrip PASSED
tests/test_record.py::test_wire_format_matches_spec PASSED
tests/test_record.py::test_modified_ciphertext_rejected PASSED
tests/test_record.py::test_task1_bitflip_attack_rejected PASSED
tests/test_record.py::test_modified_tag_rejected PASSED
tests/test_record.py::test_modified_message_type_rejected PASSED
tests/test_record.py::test_modified_version_rejected PASSED
tests/test_record.py::test_modified_length_field_rejected PASSED
tests/test_record.py::test_modified_sequence_field_rejected PASSED
tests/test_record.py::test_truncated_record_rejected PASSED
tests/test_record.py::test_replayed_record_rejected PASSED
tests/test_record.py::test_reordered_or_dropped_record_rejected PASSED

```

Figure 6: python -m pytest -v (first part).

```

ntu Tamu [Running] - Oracle VirtualBox
Oct 4 17:53
pradeep@pradeep: ~/csce465-agentsec/hw2

tests/test_handshake.py::test_kdf_matches_assignment_spec_independently PASSED
tests/test_handshake.py::test_values_are_384_byte_left_padded PASSED
tests/test_handshake.py::test_transcript_is_length_prefixed_and_unambiguous PASSED
tests/test_handshake.py::test_invalid_rsa_public_key_rejected PASSED
tests/test_handshake.py::test_invalid_pss_signature_rejected PASSED
tests/test_handshake.py::test_mitm_with_own_rsa_key_rejected PASSED
tests/test_handshake.py::test_changed_nonce_rejected[HS1-3] PASSED
tests/test_handshake.py::test_changed_nonce_rejected[HS2-3] PASSED
tests/test_handshake.py::test_changed_dh_public_value_rejected[HS1] PASSED
tests/test_handshake.py::test_changed_dh_public_value_rejected[HS2] PASSED
tests/test_handshake.py::test_degenerate_dh_public_value_rejected[0] PASSED
tests/test_handshake.py::test_degenerate_dh_public_value_rejected[1] PASSED
tests/test_handshake.py::test_degenerate_dh_public_value_rejected[p-1] PASSED
tests/test_handshake.py::test_degenerate_dh_public_value_rejected[p] PASSED
tests/test_handshake.py::test_unexpected_peer_identity_rejected PASSED
tests/test_handshake.py::test_reflected_signature_rejected_by_role_binding PASSED
tests/test_handshake.py::test_reflection_with_shared_identity_key_rejected PASSED
tests/test_handshake.py::test_reflected_dh_value_and_nonce_rejected PASSED
tests/test_handshake.py::test_reflected_hsi_back_to_gateway_rejected PASSED
tests/test_handshake.py::test_transcript_with_bad_declared_length_rejected_before_hashing PASSED
tests/test_handshake.py::test_truncated_handshake_message_rejected[1] PASSED
tests/test_handshake.py::test_truncated_handshake_message_rejected[3] PASSED
tests/test_handshake.py::test_truncated_handshake_message_rejected[50] PASSED
tests/test_handshake.py::test_trailing_bytes_rejected PASSED
tests/test_handshake.py::test_out_of_order_message_rejected PASSED
tests/test_record.py::test_valid_handshake_and_bidirectional_messages PASSED
tests/test_record.py::test_empty_plaintext_roundtrip PASSED
tests/test_record.py::test_wire_format_matches_spec PASSED
tests/test_record.py::test_modified_ciphertext_rejected PASSED
tests/test_record.py::test_task1_bitflip_attack_rejected PASSED
tests/test_record.py::test_modified_tag_rejected PASSED
tests/test_record.py::test_modified_message_type_rejected PASSED
tests/test_record.py::test_modified_version_rejected PASSED
tests/test_record.py::test_modified_length_field_rejected PASSED
tests/test_record.py::test_modified_sequence_field_rejected PASSED
tests/test_record.py::test_truncated_record_rejected PASSED
tests/test_record.py::test_replayed_record_rejected PASSED
tests/test_record.py::test_reordered_or_dropped_record_rejected PASSED
tests/test_record.py::test_record_from_other_session_rejected PASSED
tests/test_record.py::test_reflected_record_rejected PASSED
tests/test_record.py::test_reflected_record_with_rewritten_direction_rejected PASSED
tests/test_record.py::test_verify_before_decrypt_and_no_plaintext_on_error PASSED
tests/test_record.py::test_live_unique_and_sequence_never_wraps PASSED
tests/test_record.py::test_states_cannot_be_swapped PASSED
tests/test_record.py::test_spec_iv_layout_counter_overlap_is_documented PASSED

===== 49 passed in 1.28s =====
(.venv) pradeep@pradeep: ~/csce465-agentsec/hw2

```

Figure 7: python -m pytest -v (end): 49 passed.

6.2 Do the Tests Catch Real Bugs? (Mutation Check)

To check that the tests actually catch bugs, I disabled one security check at a time, ran `pytest`, looked at which tests failed, and then restored the code with `git checkout`.

Defense removed	Code change	Result
Sequence / replay check	<code>if seq != state.seq: → if False:</code>	3 tests failed
Node-signature verification	<code>verify(...ROLE_NODE...) → pass</code>	7 tests failed

```

Oct 4 17:55
pradeep@pradeep: ~/csce465-agentsec/hw2

if version != VERSION:
    raise MalformedRecord(f"unsupported version {version}")
if direction != state.direction:
    raise WrongDirection(f"got direction {direction}, expected {state.direction}")
if False:
    raise ReplayOrReorder(f"got sequence {seq}, expected {state.seq}")

ct = record[HEADER_LEN:HEADER_LEN + ct_len]
tag = record[HEADER_LEN + ct_len:]
iv = _iv(state.session_id, seq)
try:
    _mac(state.mac_key, header, iv, ct).verify(tag) # constant-time
except InvalidSignature:
    raise BadMAC('record authentication failed') from None
> secure_record.BadMAC: record authentication failed
E

secure_record.py:174: BadMAC

===== test_replayed_record_rejected =====

chan = {'g_send': <secure_record.SendState object at 0x7a0a6fd2c3b0>, 'g_recv': <secure_record.RecvState object at 0x7a0a6fd2... <secure_record.SendState object at 0x7a0a6fd07110>, 'n_recv':
re_record.RecvState object at 0x7a0a6fd04ad0>, ...}

def test_replayed_record_rejected(chan):
    r = sr.seal(chan['g_send'], sr.MSG_TOOL_CALL, CMD)
    assert sr.open_record(chan['n_recv'], r)[1] == CMD
> with pytest.raises(sr.ReplayOrReorder, match="expected 1"):
E     Failed: DID NOT RAISE ReplayOrReorder

tests/test_record.py:120: Failed

===== test_reordered_or_dropped_record_rejected =====

chan = {'g_send': <secure_record.SendState object at 0x7a0a6fd61280>, 'g_recv': <secure_record.RecvState object at 0x7a0a6fd6... <secure_record.SendState object at 0x7a0a6fd61fa0>, 'n_recv':
re_record.RecvState object at 0x7a0a6fd62d80>, ...}

def test_reordered_or_dropped_record_rejected(chan):
    sr.seal(chan['g_send'], sr.MSG_DATA, b'first') # dropped by attacker
    r1 = sr.seal(chan['g_send'], sr.MSG_DATA, b'second')
> with pytest.raises(sr.ReplayOrReorder, match="expected 0"):
E     Failed: DID NOT RAISE ReplayOrReorder

tests/test_record.py:127: Failed

===== short test summary info =====
FAILED tests/test_record.py::test_modified_sequence_field_rejected - secure_record.BadMAC: record authentication failed
FAILED tests/test_record.py::test_replayed_record_rejected - Failed: DID NOT RAISE ReplayOrReorder
FAILED tests/test_record.py::test_reordered_or_dropped_record_rejected - Failed: DID NOT RAISE ReplayOrReorder
3 failed, 46 passed in 1.33s
Updated 1 path from the index
(.venv) pradeep@pradeep: ~/csce465-agentsec/hw2

```

Figure 8: Sequence check disabled: the replay is accepted, so 3 tests fail.

```

Oct 4 17:56
pradeep@pradeep:~/csce465-agentsec/hw2
bo*\x19\x0b\x02[9\x1e\xdaE.j\x4', session_id=b'e\x07\x17\xbe\x1f\x04\xdc') is None)
E + where SessionKeys(g2n_enc=b'\x1f\xcc\x1b\xdc\t\x97\x81(\xaa\x07\x9e\xbe\x9d\x93\x92\xca\x03:0I\xaa\x0b5#\x16x2\x1d\x8f', ...f\x06\x08\x08\xfb\x0a\xee\x02\x04*<b\xabo*\x19\
2[9\x1e\xdaE.j\x4', session_id=b'e\x07\x17\xbe\x1f\x04\xdc') = <handshake.Gateway object at 0x718c697cf560>.keys

tests/test_handshake.py:146: AssertionError

test_changed_dh_public_value_rejected[H52]

params = <cryptography.hazmat.bindings._rust.openssl.DHParameters object at 0x718c699de7d0>, pair = (<handshake.Gateway object at 0x718c697ef110>, <handshake.Node object at 0x718c697ef650>
msg = 'H52'

@pytest.mark.parametrize("msg", ["H51", "H52"])
def test_changed_dh_public_value_rejected(params, pair, msg):
    gw, nd = pair
    attacker_pub = hs.int_to_fixed(params.generate_private_key()
                                   .public_key().public_numbers().y)
    n_fields = 4 if msg == 'H51' else 5

    def tamper(name, n):
        if name == msg:
            f = fields(n, n_fields)
            f[2] = attacker_pub
            return hs.encode_fields(f)
        return n

    with pytest.raises(hs.BadSignature):
        hs.run_handshake(gw, nd, relay=tamper)
> assert gw.keys is None and nd.keys is None
E AssertionError: assert (SessionKeys(g2n_enc=b'\xf8(b\xda\x99\xcc2\x92\xda4\x0b\xdb\x02Y\xfa\x7\x010\xaa_\xc81d*\xf8\x1eY\xfb\x01', g2n_...ca\xac7\x03\xfb\x03\x06\x16<\rl\x00Y\x1b\xfb\x10]
3\x010]\xbbb+\xc5\x14g\xfcC0_c\x02', session_id=b'W\x05[\x0f\x0f\x06\xfb') is None)
E + where SessionKeys(g2n_enc=b'\xf8(b\xda\x99\xcc2\x92\xda4\x0b\xdb\x02Y\xfa\x7\x010\xaa_\xc81d*\xf8\x1eY\xfb\x01', g2n_...ca\xac7\x03\xfb\x03\x06\x16<\rl\x00Y\x1b\xfb\x10]
xc6\x14g\xfcC0_c\x02', session_id=b'W\x05[\x0f\x0f\x06\xfb') = <handshake.Gateway object at 0x718c697ef110>.keys

tests/test_handshake.py:146: AssertionError

===== short test summary info =====
FAILED tests/test_handshake.py::test_incorrect_rsa_public_key_rejected - Failed: DID NOT RAISE BadSignature
FAILED tests/test_handshake.py::test_invalid_pss_signature_rejected - Failed: DID NOT RAISE BadSignature
FAILED tests/test_handshake.py::test_mitm_with_own_rsa_key_rejected - Failed: DID NOT RAISE BadSignature
FAILED tests/test_handshake.py::test_changed_nonce_rejected[H51-3] - assert (SessionKeys(g2n_enc=b'D\x05\x03k\x1d1k\x07\x08\x01\xff"\xb8:5\x01\xfa93\x04\x05\x04\x08-4H\xcb\x15\x07\x06\x15\xfa)
..n\x0c\x1c\x1d_\x19s1\x1f\x04\xcc\x94\x00Y\x96\x06\x03\x03yY\x...
FAILED tests/test_handshake.py::test_changed_nonce_rejected[H52-3] - AssertionError: assert (SessionKeys(g2n_enc=b'\xe2$\xf9w\x0e\x05\x0d0\x06\x14\x08L\x03\x0c\x01\x0a*5\x05L\x0f\x09b\x0b\x03
x082\x0e\x198V'...7\xfc\x04\x16\x02\x01)\x03\x10b\x0c\x0f\x0d\x09Y\x...
FAILED tests/test_handshake.py::test_changed_dh_public_value_rejected[H51] - AssertionError: assert (SessionKeys(g2n_enc=b'\x1f\xcc\x1b\xdc\t\x97\x81(\xaa\x07\x9e\xbe\x9d\x93\x92\xca\x03:0
xb4b5#\x16x2\x1d\x8f', ...f\x06\x08\x08\xfb\x0a\xee\x02\x04*<b\xabo*...
FAILED tests/test_handshake.py::test_changed_dh_public_value_rejected[H52] - AssertionError: assert (SessionKeys(g2n_enc=b'\xf8(b\xda\x99\xcc2\x92\xda4\x0b\xdb\x02Y\xfa\x7\x010\xaa_\xc
xf8\x1eY\xfb\x01', g2n_...ca\xac7\x03\xfb\x03\x06\x16<\rl\x00Y\x1b\xfb\x10]
7 failed, 42 passed in 1.45s
Updated 1 path from the index
(.venv) pradeep@pradeep:~/csce465-agentsec/hw2$

```

Figure 9: Signature verification disabled: MITM and tampering go undetected, so 7 tests fail.

What the two runs showed:

- **Sequence check disabled:** a replayed record was accepted. The record was genuine when it was first sent, so its MAC is still valid; a MAC proves integrity but not freshness. Also, `test_modified_sequence_field_rejected`, which flips a bit in the sequence field of a fresh record, now failed with `BadMAC` instead of `ReplayOrReorder`. The sequence number is part of both the MAC'd header and the IV, so the tampering is still caught by the MAC (defense in depth).
- **Signature verification disabled:** the gateway accepted the attacker's Diffie–Hellman value without any authentication, and 7 tests failed immediately (wrong RSA key, invalid PSS signature, MITM with Mallory's own key, changed nonces and changed DH values). This shows the tests do detect identity spoofing and transcript tampering.

6.3 Security Note

Attacker model and non-goals. I assume a standard Dolev–Yao active network attacker. The attacker can intercept, read, drop, delay, reorder, replay, modify, reflect and inject any bytes, in both the handshake and the record layer. The endpoints are assumed to be uncompromised, the long-term RSA private keys stay secret, public keys and identities are distributed correctly in advance, and the random number generator works. Out of scope are: compromised endpoints, deciding whether a command is safe, hiding message lengths or timing (traffic analysis), denial of service, post-quantum security, and re-keying.

Why CTR needs unique IVs. CTR mode creates a keystream by encrypting successive counter blocks. If any counter block is reused under the same key, the keystream repeats and the scheme becomes a two-time pad: XORing the two ciphertexts gives the XOR of the plaintexts ($C_1 \oplus C_2 = P_1 \oplus P_2$). If one plaintext is known, or has predictable header fields, the other is exposed. My design uses a fresh session identifier per handshake and a sequence number that never repeats or wraps, but as shown in Section 5.4, unique starting IVs are not enough: multi-block records must also not overlap in counter values.

Why verify before decrypt. In encrypt-then-MAC, the tag is checked before any ciphertext is decrypted. Decrypting unauthenticated data can create side channels, such as padding oracles or parsing errors, that an attacker can use to learn plaintext. Checking the MAC first rejects forged records cheaply and ensures that unauthenticated bytes never reach the decryption code, the parser or the error messages.

Why independent keys. The security proofs for encrypt-then-MAC assume that the encryption key and the MAC key are independent. Using one key for both AES-CTR and HMAC-SHA-256 breaks that assumption and can lead to unexpected interactions between the two algorithms. Separate keys for encryption and authentication, and a separate pair for each direction, limit the damage if one key is compromised and also stop reflected records at the MAC check.

Comparison with AES-GCM. AEAD modes such as AES-GCM provide confidentiality and integrity in a single operation, with one key and the header passed as associated data. GCM is also faster thanks to hardware support (AES-NI and carry-less multiplication). However, GCM is very fragile under nonce reuse: repeating one nonce leaks the authentication subkey H and allows forgeries. HMAC-SHA-256 also gives a 256-bit tag compared with GCM’s 128-bit tag. My construction needs two passes over the data, and modern protocols such as TLS 1.3 use AEAD because it is simpler to deploy correctly and fast.

Why a valid record can still be dangerous. An authenticated channel guarantees who sent a record and that it was not modified, but it says nothing about whether the command is safe or intended. A valid HMAC only proves that the sender holds the session key; it cannot tell whether that sender was compromised or manipulated through prompt injection. A compromised gateway could seal a well-formed `{"action":"DELETE","path":"/"}` and the node would open it without any error. I saw this in HW1: my OpenClaw agent ran a shell command after the prompt merely *claimed* “no approval required”. The channel was authentic; the decision was wrong. Cryptography secures the channel, so the receiving node still has to enforce authorization: an allow-list of actions and paths, least privilege, sandboxing, and human confirmation for destructive operations.

7 How to Reproduce

```
cd ~/csce465-agentsec/hw2 && source ../.venv/bin/activate
python baseline_ctr.py          # Task 1
python handshake.py             # Task 2
```

```
python secure_record.py      # Task 3
python -m pytest -v          # Task 4  (49 passed)
bash capture_evidence.sh     # regenerates evidence/*.txt
```

8 AI Use

AI assistance is described in `AI_USAGE.md`, and the exported conversation log is in `ai_logs/`. I reviewed, ran and verified all code on my VM, and I can explain every part of the design and results described in this report.

A Captured Evidence (text)

Generated on my VM by `capture_evidence.sh`. The first line of each file records the date, `user@host` and directory.

A.1 Lab preparation

```
# 2026-10-04 17:57:47 UTC | pradeep@pradeep | /home/pradeep/csce465-agentsec/hw2
$ env

Python 3.12.3
OpenSSL 3.0.13 30 Jan 2024 (Library: OpenSSL 3.0.13 30 Jan 2024)
Name: cryptography
Version: 49.0.0
Name: pytest
Version: 9.1.1

$ openssl dhparam -in ffdhe3072.pem -text -noout | head -3
    DH Parameters: (3072 bit)
    GROUP: ffdhe3072
```

A.2 Task 1

```
# 2026-10-04 17:57:47 UTC | pradeep@pradeep | /home/pradeep/csce465-agentsec/hw2
$ python baseline_ctr.py

=== Task 1: AES-256-CTR without a MAC ===
plaintext P      : {"action": "READ", "path": "notes.txt"}
nonce            : 1310681192393f53fe15e3695aa3e49a
ciphertext C     : bda586da62db96d9d42e522eaa73138af21d4fc39b1a0f547b5a7fcf62315382594eb42e

--- Attack 1: bit flip by relay (key never used) ---
offset of action : 11
modified C'      : bda586da62db96d9d42e522ba662128af21d4fc39b1a0f547b5a7fcf62315382594eb42e
receiver got     : {"action": "WIPE", "path": "notes.txt"}

XOR relation on the action bytes (C xor C' == P xor P'):
  C [11:15]      = 2eaa7313
  C' [11:15]     = 2ba66212
  C xor C'       = 050c1101
  P xor P'       = 050c1101 ('READ' xor 'WIPE')
  full-length check C xor C' == P xor P' : True
  bytes outside the action unchanged     : True

--- Attack 2: replay of the original ciphertext ---
  processed #1: {"action": "READ", "path": "notes.txt"}
  processed #2: {"action": "READ", "path": "notes.txt"}
receiver executed the same command 2 times
```

A.3 Task 2

```
# 2026-10-04 17:57:47 UTC | pradeep@pradeep | /home/pradeep/csce465-agentsec/hw2
$ python handshake.py
```

```

=== Task 2: authenticated ffdhe3072 handshake ===
group: ffdhe3072, p bits = 3072, g = 2
  HS1: 429 bytes on the wire
  HS2: 814 bytes on the wire
  HS3: 395 bytes on the wire
gateway state=established, node state=established
TH      = ea065d8262c897558c1c6db412d0b9a6296fa0e6d4c3ca145370c49501be028a
session_id = b7c8222554234070
both sides derived identical keys: True
4 traffic keys pairwise distinct : True
second session (same RSA keys) -> new session_id 92532c3381539dae, keys differ: True

```

A.4 Task 3

```

# 2026-10-04 17:57:49 UTC | pradeep@pradeep | /home/pradeep/csce465-agentsec/hw2
$ python secure_record.py

=== Task 3: encrypt-then-MAC record layer ===
G->N record #0: 83 bytes, header=01010000000000000020000000024
  node opened : (32, b'{"action":"READ","path":"notes.txt"}')
N->G record #0: 73 bytes, header=0102000000000000000210000001a
  gateway opened: (33, b'{"ok":true,"data":"hello"}')
  replay G->N #0: rejected -> ReplayOrReorder: got sequence 0, expected 1
  reflect N->G record back to node: rejected -> WrongDirection: got direction 2, expected 1
  Task-1 bit-flip on protected record: rejected -> BadMAC

```

A.5 Task 3 – OpenSSL check

```

# 2026-10-04 17:57:49 UTC | pradeep@pradeep | /home/pradeep/csce465-agentsec/hw2
$ inspect_record.py + openssl

header      = 01010000000000000020000000024
  version=01 direction=01 seq=0000000000000000 type=20 ct_len=000000024
iv          = 80fa7663890c51530000000000000000 (session_id || seq)
openssl decrypt -> {"action":"READ","path":"notes.txt"}
openssl HMAC  = d4616cf82d0b79d9ba6b60def8562c8fff0fb8c116d34ddb4b3be308e1000112
record tag    = d4616cf82d0b79d9ba6b60def8562c8fff0fb8c116d34ddb4b3be308e1000112
MATCH: tag verified independently

```

A.6 Task 4 – pytest

```

# 2026-10-04 17:57:50 UTC | pradeep@pradeep | /home/pradeep/csce465-agentsec/hw2
$ python -m pytest -v

===== test session starts =====
platform linux -- Python 3.12.3, pytest-9.1.1, pluggy-1.6.0 -- /home/pradeep/csce465-agentsec/.
  venv/bin/python
cachedir: .pytest_cache
rootdir: /home/pradeep/csce465-agentsec/hw2
collecting ... collected 49 items

tests/test_baseline.py::test_bitflip_changes_action_without_key PASSED [ 2%]
tests/test_baseline.py::test_replay_is_processed_twice PASSED [ 4%]

```

```

tests/test_handshake.py::test_valid_handshake_both_sides_agree PASSED [ 6%]
tests/test_handshake.py::test_every_session_uses_fresh_dh_and_nonces PASSED [ 8%]
tests/test_handshake.py::test_kdf_matches_assignment_spec_independently PASSED [ 10%]
tests/test_handshake.py::test_values_are_384_byte_left_padded PASSED [ 12%]
tests/test_handshake.py::test_transcript_is_length_prefixed_and_unambiguous PASSED [ 14%]
tests/test_handshake.py::test_incorrect_rsa_public_key_rejected PASSED [ 16%]
tests/test_handshake.py::test_invalid_pss_signature_rejected PASSED [ 18%]
tests/test_handshake.py::test_mitm_with_own_rsa_key_rejected PASSED [ 20%]
tests/test_handshake.py::test_changed_nonce_rejected[HS1-3] PASSED [ 22%]
tests/test_handshake.py::test_changed_nonce_rejected[HS2-3] PASSED [ 24%]
tests/test_handshake.py::test_changed_dh_public_value_rejected[HS1] PASSED [ 26%]
tests/test_handshake.py::test_changed_dh_public_value_rejected[HS2] PASSED [ 28%]
tests/test_handshake.py::test_degenerate_dh_public_value_rejected[0] PASSED [ 30%]
tests/test_handshake.py::test_degenerate_dh_public_value_rejected[1] PASSED [ 32%]
tests/test_handshake.py::test_degenerate_dh_public_value_rejected[p-1] PASSED [ 34%]
tests/test_handshake.py::test_degenerate_dh_public_value_rejected[p] PASSED [ 36%]
tests/test_handshake.py::test_unexpected_peer_identity_rejected PASSED [ 38%]
tests/test_handshake.py::test_reflected_signature_rejected_by_role_binding PASSED [ 40%]
tests/test_handshake.py::test_reflection_with_shared_identity_key_rejected PASSED [ 42%]
tests/test_handshake.py::test_reflected_dh_value_and_nonce_rejected PASSED [ 44%]
tests/test_handshake.py::test_reflected_hs1_back_to_gateway_rejected PASSED [ 46%]
tests/test_handshake.py::test_transcript_with_bad_declared_length_rejected_before_hashing PASSED
[ 48%]
tests/test_handshake.py::test_truncated_handshake_message_rejected[1] PASSED [ 51%]
tests/test_handshake.py::test_truncated_handshake_message_rejected[3] PASSED [ 53%]
tests/test_handshake.py::test_truncated_handshake_message_rejected[50] PASSED [ 55%]
tests/test_handshake.py::test_trailing_bytes_rejected PASSED [ 57%]
tests/test_handshake.py::test_out_of_order_message_rejected PASSED [ 59%]
tests/test_record.py::test_valid_handshake_and_bidirectional_messages PASSED [ 61%]
tests/test_record.py::test_empty_plaintext_roundtrip PASSED [ 63%]
tests/test_record.py::test_wire_format_matches_spec PASSED [ 65%]
tests/test_record.py::test_modified_ciphertext_rejected PASSED [ 67%]
tests/test_record.py::test_task1_bitflip_attack_rejected PASSED [ 69%]
tests/test_record.py::test_modified_tag_rejected PASSED [ 71%]
tests/test_record.py::test_modified_message_type_rejected PASSED [ 73%]
tests/test_record.py::test_modified_version_rejected PASSED [ 75%]
tests/test_record.py::test_modified_length_field_rejected PASSED [ 77%]
tests/test_record.py::test_modified_sequence_field_rejected PASSED [ 79%]
tests/test_record.py::test_truncated_record_rejected PASSED [ 81%]
tests/test_record.py::test_replayed_record_rejected PASSED [ 83%]
tests/test_record.py::test_reordered_or_dropped_record_rejected PASSED [ 85%]
tests/test_record.py::test_record_from_other_session_rejected PASSED [ 87%]
tests/test_record.py::test_reflected_record_rejected PASSED [ 89%]
tests/test_record.py::test_reflected_record_with_rewritten_direction_rejected PASSED [ 91%]
tests/test_record.py::test_verify_before_decrypt_and_no_plaintext_on_error PASSED [ 93%]
tests/test_record.py::test_ivs_unique_and_sequence_never_wraps PASSED [ 95%]
tests/test_record.py::test_states_cannot_be_swapped PASSED [ 97%]
tests/test_record.py::test_spec_iv_layout_counter_overlap_is_documented PASSED [100%]

===== 49 passed in 1.03s =====

```



```

itu Tamu [Running] - Oracle VirtualBox
Oct 4 17:56
pradeep@pradeep:~/csce465-agentsec/hw2
(.venv) pradeep@pradeep:~/csce465-agentsec/hw2$ sed -i 's/verify(self.peer_public_key, sig, ROLE_NODE, th)/pass/' handshake.py
python -m pytest -q
git checkout handshake.py
.....FFFFFFFF.....
===== FAILURES =====
test_incorrect_rsa_public_key_rejected

params = <cryptography.hazmat.bindings._rust.openssl.dh.DHParameters object at 0x718c699de7d0>
rsa_keys = {'gateway': <cryptography.hazmat.bindings._rust.openssl.rsa.RSAPrivateKey object at 0x718c699dee70>, 'node': <cryptogr...at 0x718c699de130>, 'mallory': <cryptography.hazmat.binding
st.openssl.rsa.RSAPrivateKey object at 0x718c699ddb30>}

def test_incorrect_rsa_public_key_rejected(params, rsa_keys):
    # Gateway is configured with the WRONG public key for the node.
    gw = hs.Gateway(b'gateway-01', rsa_keys['gateway'], b'node-07',
        rsa_keys['mallory'].public_key(), params)
    nd = hs.Node(b'node-07', rsa_keys['node'], b'gateway-01',
        rsa_keys['gateway'].public_key(), params)
> with pytest.raises(hs.BadSignature, match='node'):
E Failed: DID NOT RAISE BadSignature

tests/test_handshake.py:81: Failed
test_invalid_pss_signature_rejected

pair = (<handshake.Gateway object at 0x718c697ee0f0>, <handshake.Node object at 0x718c697ed220>)

def test_invalid_pss_signature_rejected(pair):
    gw, nd = pair

    def flip_sig(name, m):
        if name == 'HS2':
            f = fields(m, 5)
            f[4] = bytes([f[4][0] ^ 1]) + f[4][1:]
            return hs.encode_fields(f)
        return m

> with pytest.raises(hs.BadSignature):
E Failed: DID NOT RAISE BadSignature

tests/test_handshake.py:96: Failed
test_mitm_with_own_rsa_key_rejected

params = <cryptography.hazmat.bindings._rust.openssl.dh.DHParameters object at 0x718c699de7d0>
rsa_keys = {'gateway': <cryptography.hazmat.bindings._rust.openssl.rsa.RSAPrivateKey object at 0x718c699dee70>, 'node': <cryptogr...at 0x718c699de130>, 'mallory': <cryptography.hazmat.binding
st.openssl.rsa.RSAPrivateKey object at 0x718c699ddb30>}
pair = (<handshake.Gateway object at 0x718c697ecdd0>, <handshake.Node object at 0x718c697ecbc0>)

def test_mitm_with_own_rsa_key_rejected(params, rsa_keys, pair):
    """Mallory substitutes her DH value and signs with her own RSA key."""
    gw, nd = pair

```

Figure 11: Signature-verification mutation (first part of the failure output).